

cleaning_pokemon

August 12, 2026

0.1 # Cleaning up a dataset

When there's a new dataset to work with, the data frequently has mistakes. You have to clean it up before it can be properly used.

Let's practice doing this with a dataset containing a list of Gen I pokemon and their attributes.

```
[1]: import pandas as pd
      from functools import reduce

      pokemon = pd.read_csv('pokemon.csv')
      pokemon
```

```
[1]:      Name Height (in) Weight (lbs)      Type \
0    Bulbasaur      28      15.20  Grass, Poison
1     Ivysaur      39      28.70  Grass, Poison
2    Venusaur      79     220.50  Grass, Poison
3   Charmander      24      18.70        Fire
4  Charmeleon      43      41.90        Fire
..      ...      ...      ...      ...
149   Dratini      71       7.30        Dragon
150  Dragonair     157      36.40        Dragon
151  Dragonite      87     463.00  Dragon, Flying
152    Mewtwo      79     269.00    Psychic
153      Mew   Unknown       8.80    Psychic
```

```
      Weaknesses
0  Fire, Psychic, Flying, Ice
1  Fire, Psychic, Flying, Ice
2  Fire, Psychic, Flying, Ice
3    Water, Ground, Rock
4    Water, Ground, Rock
..      ...
149   Fairy, Ice, Dragon
150   Fairy, Ice, Dragon
151  Fairy, Dragon, Ice, Rock
152    Ghost, Dark, Bug
153    Ghost, Dark, Bug
```

[154 rows x 5 columns]

```
[2]: pokemon.shape
```

```
[2]: (154, 5)
```

```
[3]: pokemon.dtypes
```

```
[3]: Name          str
     Height (in)   str
     Weight (lbs)  str
     Type          str
     Weaknesses    str
     dtype: object
```

All the data types are strings. That's not good. Some of these should be numbers. There must be at least one string in each column.

```
[4]: pokemon.describe()
```

```
[4]:
```

	Name	Height (in)	Weight (lbs)	Type	Weaknesses
count	154	153	153	152	154
unique	151	32	110	50	49
top	Blastoise	39	66.10	Water	Grass, Electric
freq	2	19	6	15	16

0.2 Some preliminaries: a search function and a shift function and elementwise and/or functions for lists

I'd like to be able to search for multiple things across the entire dataframe at once.

I'd like to be able to fix the order of the pokemon.

I'd like to be able to do elementwise and/or on lists.

```
[5]: def search(df, ll, exclude=None):
      """
      df is a dataframe, ll is a list of search terms
      this function will search every column of the dataframe for any occurrences
      of any of the search
      terms and return a series that can be used as index for slicing a dataframe
      You can search for both numbers and strings in a single list of search terms
      the function will search the dataframe for anywhere your search strings are
      contained (containment)
      the function will search the dataframe for anywhere your search numbers
      occur (equality)
      You can also include a list of terms to exclude. it should be the same
      length as ll
      It should also be paired up with ll elementwise
```

```

    for example, ll=['Ic', 'Gras'] and exclude=['Ice', 'Grass'] will find
    ↪ instances where
        'Ic' is present, but 'Ice' isn't; and where 'Gras' is present, but 'Grass'
    ↪ isn't
        this way you can find the typos 'Ic' and 'Gras' without finding all the
    ↪ correct ones with 'Ice'
        and 'Grass'
    """
    # this returns a nested list where the outside dim=number of search terms
    ↪ in ll,
    # middle dim= number of columns in df
    # inside dim=number of rows in df
    if exclude == None:
        ind = [[[test in str(string) if type(test)==str else test==string for
    ↪ string in df[col].values]
                for col in df.columns]
                for test in ll]
    else:
        ind = [[[(ll[i] in str(string) and exclude[i] not in str(string)) if
    ↪ type(ll[i])==str else ll[i]==string for string in df[col].values]
                for col in df.columns]
                for i in range(len(ll))]
    # now we perform element wise or across the columns and search terms to
    ↪ give of the rows where
    # any of the search terms exist
    ind = [reduce(elementwise_or, ind[i]) for i in range(len(ind))] # for each
    ↪ search term, collate the columns
    ind = reduce(elementwise_or, ind) # collate the search terms
    return ind

def move_and_shift(df, start, target):
    """
    This function takes the index of the row you'd like to move and the index
    ↪ to which you'd like
        to move it
        the function will shift all of the rows in between up or down one to make
    ↪ space
        46 start
        22 target
    """
    insert = df.iloc[start]

    if start > target: # if you want to move the row up and shift the rest down:
        df.iloc[target+1:start+1] = df.iloc[target:start]
    if start < target: # if you want to move the row down and shift the rest up:

```

```

df.iloc[start:target] = df.iloc[start+1:target+1]

df.iloc[target] = insert

def elementwise_or(list1, list2):
    return [list1[i] or list2[i] for i in range(len(list1))]

def elementwise_and(list1, list2):
    return [list1[i] and list2[i] for i in range(len(list1))]

```

0.3 Data cleaning

Now that we've seen an overview of the data, it's time to clean it up: - Duplicates - Missing data - Incorrect data types (e.g., strings where they don't belong) - Bad data formatting (e.g., using the wrong case or punctuation) - Typos - Hanging Whitespace

0.3.1 Duplicates

Let's find the duplicates. Set `keep=False` to see them all, otherwise you'll only see one of each, not two.

```
[6]: pokemon[pokemon.duplicated('Name', keep=False)]
```

```
[6]:
```

	Name	Height (in)	Weight (lbs)	Type	Weaknesses
8	Blastoise	63 inches	188.50	Water	Grass, Electric
9	Blastoise	63	188.50	Water	Grass, Electric
23	Ekans	79	15.20	Poison	Psychic, Ground
48	Ekans	79	15.20	Poison	Psychic, Ground
84	Farfetch'd	31	33.10	Normal, Flying	Electric, Ice, Rock
91	Farfetch'd	31	33.10	Normal, Flying	Electric, Ice, Rock

Now let's drop them. It looks like they're identical except the first Blastoise, which has a data type issue. So let's keep the last ones of each and save it inplace and reset our indices so that they don't skip.

```
[7]: pokemon.drop_duplicates('Name', keep='last', inplace=True, ignore_index=True)
pokemon
```

```
[7]:
```

	Name	Height (in)	Weight (lbs)	Type	\
0	Bulbasaur	28	15.20	Grass, Poison	
1	Ivysaur	39	28.70	Grass, Poison	
2	Venusaur	79	220.50	Grass, Poison	
3	Charmander	24	18.70	Fire	
4	Charmeleon	43	41.90	Fire	

```

..      ...      ...      ...      ...
146      Dratini      71      7.30      Dragon
147      Dragonair      157      36.40      Dragon
148      Dragonite      87      463.00      Dragon, Flying
149      Mewtwo      79      269.00      Psychic
150      Mew      Unknown      8.80      Psychic

```

```

Weaknesses
0      Fire, Psychic, Flying, Ice
1      Fire, Psychic, Flying, Ice
2      Fire, Psychic, Flying, Ice
3      Water, Ground, Rock
4      Water, Ground, Rock
..      ...
146      Fairy, Ice, Dragon
147      Fairy, Ice, Dragon
148      Fairy, Dragon, Ice, Rock
149      Ghost, Dark, Bug
150      Ghost, Dark, Bug

```

[151 rows x 5 columns]

```
[8]: pokemon[pokemon.duplicated('Name', keep=False)]
```

```
[8]: Empty DataFrame
Columns: [Name, Height (in), Weight (lbs), Type, Weaknesses]
Index: []
```

Tah-dah. No more duplicates. Now let's make sure they're in the right order by pokedex number. - Blastoise is #9 -> index 8 - Ekans is #23 -> index 22 - Farfetch'd is #83 -> index 82

```
[9]: pokemon[search(pokemon, ["Blastoise", "Ekans", "Farfetch'd"])]
```

```
[9]:      Name Height (in) Weight (lbs)      Type      Weaknesses
8      Blastoise      63      188.50      Water      Grass, Electric
46      Ekans      79      15.20      Poison      Psychic, Ground
88      Farfetch'd      31      33.10      Normal, Flying      Electric, Ice, Rock

```

```
[10]: move_and_shift(pokemon, 46, 22)
move_and_shift(pokemon, 88, 82)
pokemon[search(pokemon, ["Blastoise", "Ekans", "Farfetch'd"])]
```

```
[10]:      Name Height (in) Weight (lbs)      Type      Weaknesses
8      Blastoise      63      188.50      Water      Grass, Electric
22      Ekans      79      15.20      Poison      Psychic, Ground
82      Farfetch'd      31      33.10      Normal, Flying      Electric, Ice, Rock

```

0.3.2 Missing data

Missing data could appear in any column, and it can appear as a literal blank, or else None or NaN, etc.
isna will find it.

```
[11]: pokemon[(pd.isna(pokemon['Height (in)']) |
               pd.isna(pokemon['Weight (lbs)']) |
               pd.isna(pokemon['Type']) |
               pd.isna(pokemon['Weaknesses']))]
```

```
[11]:      Name Height (in) Weight (lbs)      Type \
41  Golbat      NaN      NaN  Poison, Flying
66  Machoke      59      155.40      NaN
85   Seel      43      198.40      NaN

      Weaknesses
41  Psychic, Electric, Ice, Rock
66      Psychic, Flying, Fairy
85      Grass, Electric
```

I checked bulbapedia. Let's enter the correct information:

Note that I have to enter the numbers as strings at this phase of cleaning because I haven't solved the issue of all the data types being strings yet.

```
[12]: pokemon.at[41, 'Height (in)'] = '63'
       pokemon.at[41, 'Weight (lbs)'] = '121.3'
       pokemon.at[66, 'Type'] = 'Fighting'
       pokemon.at[85, 'Type'] = 'Water'

       pokemon.iloc[[41, 66, 85]]
```

```
[12]:      Name Height (in) Weight (lbs)      Type \
41  Golbat      63      121.3  Poison, Flying
66  Machoke      59      155.40  Fighting
85   Seel      43      198.40      Water

      Weaknesses
41  Psychic, Electric, Ice, Rock
66      Psychic, Flying, Fairy
85      Grass, Electric
```

Tah-dah.

Another way data could be missing is if they entered 'Unknown' or 'unknown.'

```
[13]: pokemon[search(pokemon, ['Unknown', 'unknown'])]
```

```
[13]:      Name Height (in) Weight (lbs)      Type      Weaknesses
      34   Unknown      24      16.50   Fairy      Steel, Posion
      117  Goldeen      24      33.10  Unknown  Grass, Electric
      150    Mew      Unknown      8.80  Psychic  Ghost, Dark, Bug
```

Now let's fill these in. Back to Bulbapedia. - Still entering them as strings.

```
[14]: pokemon.at[34, 'Name'] = 'Clefaairy'
      pokemon.at[117, 'Type'] = 'Water'
      pokemon.at[150, 'Height (in)'] = '16'

      pokemon.iloc[[34, 117, 150]]
```

```
[14]:      Name Height (in) Weight (lbs)      Type      Weaknesses
      34  Clefaairy      24      16.50   Fairy      Steel, Posion
      117  Goldeen      24      33.10   Water  Grass, Electric
      150    Mew      16      8.80  Psychic  Ghost, Dark, Bug
```

0.3.3 Incorrect data types

Now let's make height and weight into numbers like they're supposed to be.

```
[15]: pokemon['Height (in)'] = pd.to_numeric(pokemon['Height (in)'], errors='coerce').
      ↪ astype('Int64')
      pokemon['Weight (lbs)'] = pd.to_numeric(pokemon['Weight (lbs)'],
      ↪ errors='coerce').astype('Float64')
```

Now we've turned the numbers into numbers and the other stuff into NaN, so let's go find those NaNs and fix them like we did before.

```
[16]: pokemon[(pd.isna(pokemon['Height (in)']) |
      pd.isna(pokemon['Weight (lbs)'])
      )]
```

```
[16]:      Name Height (in) Weight (lbs)      Type \
      23    Arbok      138      <NA>      Posion
      28  Nidoran (f)      <NA>      15.4      Poison
      35  Clefable      <NA>      88.2      Fairy
      74  Graveler      <NA>     231.5  Rock, Ground
      76   Ponyta      39      <NA>      Fie
      126  Pinsir      <NA>     121.3      Bug
      142  Snorlax      83      <NA>      Normal
      145  Moltres      79      <NA>  Fire, Flying

      Weaknesses
      23      Psychic, Ground
      28      Psychic, Ground
```

```

35          Steel, Posion
74  Steel, Fighting, Water, Ice, Grass, Ground
76          Water, Ground, Rock
126         Fire, Flying, Rock
142          Fighting
145         Water, Electric, Rock

```

Back to Bulbapedia.

```

[17]: pokemon.at[28, 'Height (in)'] = 12 * 1 + 4
pokemon.at[35, 'Height (in)'] = 12 * 4 + 3
pokemon.at[74, 'Height (in)'] = 12 * 3 + 3
pokemon.at[126, 'Height (in)'] = 12 * 4 + 11
pokemon.at[23, 'Weight (lbs)'] = 143.3
pokemon.at[76, 'Weight (lbs)'] = 66.1
pokemon.at[142, 'Weight (lbs)'] = 1014.1
pokemon.at[145, 'Weight (lbs)'] = 132.3

pokemon.iloc[[23, 28, 35, 74, 76, 126, 142, 145]]

```

```

[17]:      Name  Height (in)  Weight (lbs)      Type \
23      Arbok          138         143.3      Posion
28  Nidoran (f)          16          15.4      Poison
35      Clefable          51          88.2       Fairy
74      Graveler          39        231.5  Rock, Ground
76      Ponyta          39          66.1       Fie
126     Pinsir          59        121.3       Bug
142     Snorlax          83       1014.1     Normal
145     Moltres          79        132.3  Fire, Flying

```

```

Weaknesses
23          Psychic, Ground
28          Psychic, Ground
35          Steel, Posion
74  Steel, Fighting, Water, Ice, Grass, Ground
76          Water, Ground, Rock
126         Fire, Flying, Rock
142          Fighting
145         Water, Electric, Rock

```

```

[18]: pokemon.dtypes

```

```

[18]: Name          str
Height (in)      Int64
Weight (lbs)     Float64
Type             str
Weaknesses       str
dtype: object

```

The correct values have been entered and the datatype is now correct.

0.3.4 Bad data formatting and/or typos

We expect most of these to occur in the Type and Weaknesses columns. Thankfully those columns should have a relatively small number of different values. So let's get a short list of unique values and just read them for errors.

```
[19]: # basically what i'm going to do is get the list of unique types,
      # and then join and split them from the whitespace and commas until i can see
      ↪all the types
types = pokemon.Type.unique()
types = ','.join(types)
types = types.split()
types = ','.join(types)
types = types.split(',')
types = set(types)
types.remove('')
types
```

```
[19]: {'Bug',
      'Dragon',
      'Electric',
      'Fairy',
      'Fie',
      'Fighting',
      'Fire',
      'Flying',
      'Ghost',
      'Grass',
      'Ground',
      'Ice',
      'Normal',
      'Poison',
      'Posion',
      'Psychic',
      'Rock',
      'Steel',
      'Water'}
```

Fie, and Posion are not correct. Let's find them and destroy them.

```
[20]: pokemon[search(pokemon, ['Fie', 'Posion'])]
```

```
[20]:
```

	Name	Height (in)	Weight (lbs)	Type \
23	Arbok	138	143.3	Posion
29	Nidorina	31	44.1	Posion

30	Nidoqueen	51	132.3	Posion, Ground
31	Nidoran (m)	20	19.8	Posion
34	Clefairy	24	16.5	Fairy
35	Clefable	51	88.2	Fairy
38	Jigglypuff	20	12.1	Normal, Fairy
39	Wigglytuff	39	26.5	Normal, Fairy
76	Ponyta	39	66.1	Fie

	Weaknesses
23	Psychic, Ground
29	Psychic, Ground
30	Water, Psychic, Ice, Ground
31	Psychic, Ground
34	Steel, Posion
35	Steel, Posion
38	Steel, Posion
39	Steel, Posion
76	Water, Ground, Rock

```
[21]: pokemon.at[23, 'Type'] = 'Poison'
pokemon.at[29, 'Type'] = 'Poison'
pokemon.at[30, 'Type'] = 'Poison, Ground'
pokemon.at[31, 'Type'] = 'Poison'
pokemon.at[34, 'Weaknesses'] = 'Steel, Poison'
pokemon.at[35, 'Weaknesses'] = 'Steel, Poison'
pokemon.at[38, 'Weaknesses'] = 'Steel, Poison'
pokemon.at[39, 'Weaknesses'] = 'Steel, Poison'
pokemon.at[76, 'Type'] = 'Fire'

pokemon.iloc[[23, 29, 30, 31, 34, 35, 38, 39, 76]]
```

	Name	Height (in)	Weight (lbs)	Type \
23	Arbok	138	143.3	Poison
29	Nidorina	31	44.1	Poison
30	Nidoqueen	51	132.3	Poison, Ground
31	Nidoran (m)	20	19.8	Poison
34	Clefairy	24	16.5	Fairy
35	Clefable	51	88.2	Fairy
38	Jigglypuff	20	12.1	Normal, Fairy
39	Wigglytuff	39	26.5	Normal, Fairy
76	Ponyta	39	66.1	Fire

	Weaknesses
23	Psychic, Ground
29	Psychic, Ground
30	Water, Psychic, Ice, Ground
31	Psychic, Ground

```

34             Steel, Poison
35             Steel, Poison
38             Steel, Poison
39             Steel, Poison
76             Water, Ground, Rock

```

Tah-dah.

Now let's do the same thing for weaknesses.

```

[22]: # basically what i'm going to do is get the list of unique types,
      # and then join and split them from the whitespace and commas until i can see
      ↪ all the types
weak = ','.join(pokemon.Weaknesses.unique())
weak = weak.split()
weak = ','.join(weak)
weak = weak.split(',')
weak = set(weak)
weak.remove('')
weak

```

```

[22]: {'-',
      'Bug',
      'Dark',
      'Dragon',
      'Electric',
      'Fairy',
      'Fighting',
      'Fire',
      'Flying',
      'Flyng',
      'Ghost',
      'Gras',
      'Grass',
      'Ground',
      'Ic',
      'Ice',
      'Ie',
      'Poison',
      'Psychic',
      'Rock',
      'Steel',
      'Water'}

```

Flyng, Gras, Ic, Ie are incorrect. Also, there shouldn't be any dashes. Commas only. Let's go get them.

```

[23]: pokemon[search(pokemon, ['Flyng', 'Gras,', 'Ic', 'Ie', '-'])]

```

[23] :	Name	Height (in)	Weight (lbs)	Type \
0	Bulbasaur	28	15.2	Grass, Poison
1	Ivysaur	39	28.7	Grass, Poison
2	Venusaur	79	220.5	Grass, Poison
6	Squirtle	20	19.8	Water
11	Butterfree	43	70.5	Bug, Flying
15	Pidgey	12	4.0	Normal, Flying
16	Pidgeotto	43	66.1	Normal, Flying
17	Pidgeot	59	87.1	Normal, Flying
20	Spearow	12	4.4	Normal, Flying
21	Fearow	47	83.8	Normal, Flying
26	Sandsshrew	24	26.5	Ground
27	Sandslash	39	65.0	Ground
30	Nidoqueen	51	132.3	Poison, Ground
33	Nidoking	55	136.7	Poison, Ground
40	Zubat	31	16.5	Poison, Flying
41	Golbat	63	121.3	Poison, Flying
42	Oddish	20	11.9	Grass, Poison
43	Gloom	31	19.0	Grass, Poison
44	Vileplume	47	41.0	Grass, Poison
45	Paras	12	11.9	Bug, Grass
46	Parasect	39	65.0	Bug, Grass
49	Diglett	8	1.8	Ground
50	Dugtrio	28	73.4	Ground
53	Psyduck	31	43.2	Water
56	Primeape	39	70.5	Fighting
64	Alakazam	59	-105.8	Psychic
68	Bellsprout	16	8.8	Grass, Poison
69	Weepinbell	39	14.1	Grass, Poison
70	Victreebell	67	34.2	Grass, Poison
73	Geodude	16	44.1	Rock, Ground
74	Graveler	39	231.5	Rock, Ground
75	Golem	55	661.4	Rock, Ground
82	Farfetch'd	31	33.1	Normal, Flying
83	Doduo	55	86.4	Normal, Flying
84	Dodrio	71	187.8	Normal, Flying
86	Dewgong	67	264.6	Water, Ice
90	Cloyster	59	292.1	Water, Ice
94	Onix	346	463.0	Rock, Ground
101	Exeggcute	16	5.5	Grass, Psychic
102	Exeggutor	79	264.6	Grass, Psychic
103	Cubone	16	14.3	Ground
104	Marowak	39	99.2	Ground
107	Lickitung	47	-144.4	Normal
110	Rhyhorn	39	253.5	Ground, Rock
111	Rhydon	75	264.6	Ground, Rock
113	Tangela	39	77.2	Grass

122	Scyther	59	123.5	Bug, Flying
123	Jynx	55	89.5	Ice, Psychic
130	Lapras	98	485.0	Water, Ice
141	Aerodactyl	71	130.1	Rock, Flying
143	Articuno	67	122.1	Ice, Flying
144	Zapdos	63	116.0	Electric, Flying
146	Dratini	71	7.3	Dragon
147	Dragonair	157	36.4	Dragon
148	Dragonite	87	463.0	Dragon, Flying

Weaknesses

0	Fire, Psychic, Flying, Ice
1	Fire, Psychic, Flying, Ice
2	Fire, Psychic, Flying, Ice
6	Grass, Electric
11	Fire, Flying, Electric, Ice, Rock
15	Electric, Ice, Rock
16	Electric, Ice, Rock
17	Electric, Ice, Rock
20	Electric, Ice, Rock
21	Electric, Ice, Rock
26	Water, Grass, Ice
27	Water, Grass, Ice
30	Water, Psychic, Ice, Ground
33	Water, Psychic, Ice, Ground
40	Psychic, Electric, Ice, Rock
41	Psychic, Electric, Ice, Rock
42	Fire, Psychic, Flying, Ice
43	Fire, Psychic, Flying, Ice
44	Fire, Psychic, Flying, Ice
45	Fire, Flying, Ice, Poison, Rock, Bug
46	Fire, Flying, Ice, Poison, Rock, Bug
49	Water, Grass, Ice
50	Water, Grass, Ice
53	Grass - Electric
56	Psychic, Flying, Fairy
64	Ghost, Dark, Bug
68	Fire, Psychic, Flying, Ice
69	Fire, Psychic, Flying, Ice
70	Fire, Psychic, Flying, Ice
73	Steel, Fighting, Water, Ice, Grass, Ground
74	Steel, Fighting, Water, Ice, Grass, Ground
75	Steel, Fighting, Water, Ice, Grass, Ground
82	Electric, Ice, Rock
83	Electric, Ice, Rock
84	Electric, Ice, Rock
86	Grass, Electric, Flying, Rock

```

90             Grass, Electric, Fighting, Rock
94         Steel, Fighting, Water, Ice, Grass, Ground
101  Ghost - Fire - Flying - Ice - Dark - Poison - Bug
102         Ghost, Fire, Flying, Ice, Dark, Poison, Bug
103             Water, Grass, Ice
104             Water, Grass, Ice
107                 Fighting
110         Steel, Ice, Water, Fighting, Grass, Ground
111         Steel, Ice, Water, Fighting, Grass, Ground
113             Fire, Flying, Ice, Poison, Bug
122             Fire, Flying, Electric, Ice, Rock
123         Steel, Ghost, Fire, Dark, Rock, Bug
130             Grass, Electric, Fighting, Rock
141         Steel, Water, Electric, Ice, Rock
143             Steel, Fire, Electric, Rock
144                 Ice, Rock
146                 Fairy, Ice, Dragon
147                 Fairy, Ice, Dragon
148                 Fairy, Dragon, Ice, Rock

```

Well that's not good. As we might have suspected, searching for 'Ic' returns every ice type because 'Ic' is contained in 'Ice'. Similar for 'Gras'. Let's just search for the other terms and hold off on 'Ic' and 'Gras'.

```
[24]: pokemon[search(pokemon, ['Flyng', 'Ie', '-'])]
```

```
[24]:
```

	Name	Height (in)	Weight (lbs)	Type \
0	Bulbasaur	28	15.2	Grass, Poison
20	Spearow	12	4.4	Normal, Flying
53	Psyduck	31	43.2	Water
56	Primeape	39	70.5	Fighting
64	Alakazam	59	-105.8	Psychic
101	Exeggcute	16	5.5	Grass, Psychic
107	Lickitung	47	-144.4	Normal

```

                                Weaknesses
0                                Fire, Psychic, Flying, Ie
20                               Electric, Ie, Rock
53                               Grass - Electric
56                               Psychic, Flyng, Fairy
64                               Ghost, Dark, Bug
101  Ghost - Fire - Flying - Ice - Dark - Poison - Bug
107                               Fighting

```

There we go. And we also caught some bad negative numbers too! Let's fix those while we're at it.

```
[25]: pokemon.at[0, 'Weaknesses'] = 'Fire, Psychic, Flying, Ice'
pokemon.at[20, 'Weaknesses'] = 'Electric, Ice, Rock'
pokemon.at[53, 'Weaknesses'] = 'Grass, Electric'
pokemon.at[56, 'Weaknesses'] = 'Psychic, Flying, Fairy'
pokemon.at[64, 'Weight (lbs)'] = 105.8
pokemon.at[101, 'Weaknesses'] = 'Ghost, Fire, Flying, Ice, Dark, Poison, Bug'
pokemon.at[107, 'Weight (lbs)'] = 144.4

pokemon.iloc[[0, 20, 53, 56, 64, 101, 107]]
```

```
[25]:
```

	Name	Height (in)	Weight (lbs)	Type \
0	Bulbasaur	28	15.2	Grass, Poison
20	Spearow	12	4.4	Normal, Flying
53	Psyduck	31	43.2	Water
56	Primeape	39	70.5	Fighting
64	Alakazam	59	105.8	Psychic
101	Exeggcute	16	5.5	Grass, Psychic
107	Lickitung	47	144.4	Normal

	Weaknesses
0	Fire, Psychic, Flying, Ice
20	Electric, Ice, Rock
53	Grass, Electric
56	Psychic, Flying, Fairy
64	Ghost, Dark, Bug
101	Ghost, Fire, Flying, Ice, Dark, Poison, Bug
107	Fighting

Now let's go back and find and fix 'Ic' and 'Gras', excluding the good ones.

```
[26]: pokemon[search(pokemon, ['Ic', 'Gras'], exclude=['Ice', 'Grass'])]
```

```
[26]:
```

	Name	Height (in)	Weight (lbs)	Type	Weaknesses
6	Squirtle	20	19.8	Water	Gras, Electric
16	Pidgeotto	43	66.1	Normal, Flying	Electric, Ic, Rock

```
[27]: pokemon.at[6, 'Weaknesses'] = 'Grass, Electric'
pokemon.at[16, 'Weaknesses'] = 'Electric, Ice, Rock'

pokemon.iloc[[6, 16]]
```

```
[27]:
```

	Name	Height (in)	Weight (lbs)	Type	Weaknesses
6	Squirtle	20	19.8	Water	Grass, Electric
16	Pidgeotto	43	66.1	Normal, Flying	Electric, Ice, Rock

0.3.5 Hanging Whitespace

One last thing.

```
[28]: pokemon = pokemon.map(lambda x: x.strip() if isinstance(x, str) else x)
```

0.4 The end.

And that should do it. Let's save our changes to a .csv and go look at it in excel to verify.

```
[29]: pokemon.to_csv('clean_pokemon.csv', index=False)
```